EMBEDDED SYSTEM WITH IOT
EXPERIMENT NO. 7

IoT based Fire Alarm System

Name: NAVARRO, ROD GERYK C.

Course/Section: CPE162P-4/E01

Date of Performance: 06/16/2025
Date of Submission: 06/20/2025

CYREL O. MANLISES, PH.D.
Instructor

DISCUSSION

Connections

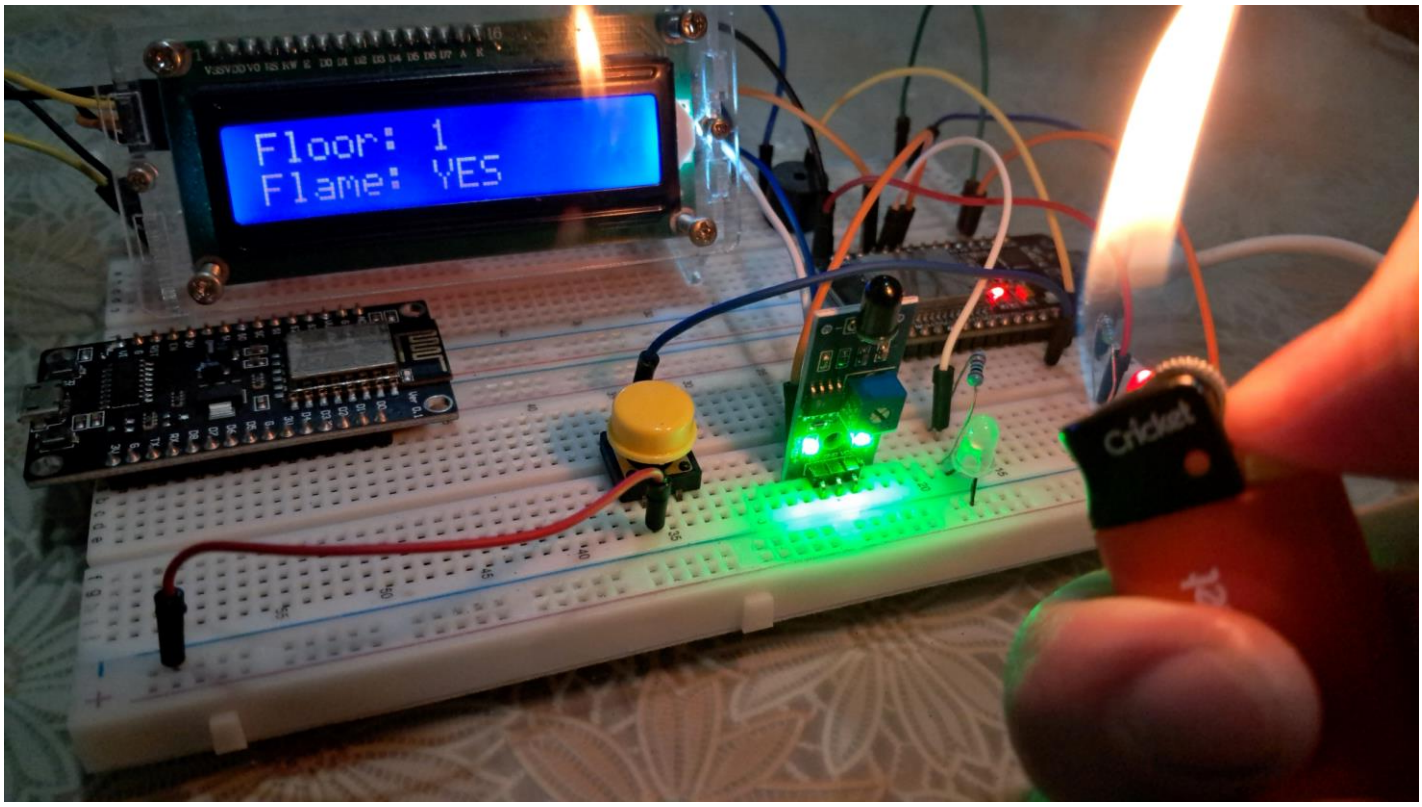


FIGURE 1: The Connection of the Flame Sensor, LEDs, Buzzer, and LCD to the ESP32 Dev Module

This project aims to monitor fire incidents in each floor of the building, helping to identify which floor is prone to fire incidents. The system uses a flame sensor module to detect near fire. Based on the status of each floors, two LEDs are used to indicate the presence of a fire in the floor detected by the sensor. The system also displays the current floor level of the building (assuming a five-story building) and status of each floor (fire detected or no fire detected) on an LCD and sends the data to ThingSpeak for cloud monitoring. In addition, a button is utilized to switch floor levels in order to input sample conditions in each floor and a buzzer that sounds an alarm when fire is detected.

The first step was connecting all the parts to the ESP32. For monitoring and analyzing the data, I used ThingSpeak, an IoT platform that displays real-time updates. The ESP32 sends data from the flame sensor and the number of fire incidents in each floor to ThingSpeak, allowing users to monitor the the frequency of fire incidents in each floor of the building. After this, I programmed the system using the Arduino IDE to make all the components work together properly.

Code Development

Navarro_EXP7_CPE162P.ino

```
#include <LiquidCrystal_I2C.h>
#include <WiFi.h>
#include <ThingSpeak.h>

//wiFi and thingspeak config
const char* ssid = "PLDTHOMEFIBRt6ucX";
const char* password = "PLDTWIFItLK72";
const long channel = 2964917;
const char key[] = "18A6ZBU4I3OLZZ44";

// pin definitions
const int RED_LED = 23, WHITE_LED = 19, buzzer = 13, flamePin = 18;
const int floorBtn = 14;

// variables
int floorLevel = 1;
int flame = 0;
int fireCounts[6] = {0}; // fire counters
bool lastBtnState = HIGH;
unsigned long lastUpdate = 0;
WiFiClient client;
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

FIGURE 2: The Initialization of Variables and Components

In this project, I started the code by declaring all the necessary variables and settings. I included the Wi-Fi name and password so the ESP32 can connect to the internet, as well as the ThingSpeak channel number and API key to send data online. I defined the pins used for the flame sensor, two LEDs (red and white), a buzzer, and a button that changes the floor

level. I also created a variable called floorLevel to keep track of the current floor (from 1 to 5), and an array called fireCounts to record how many times fire was detected on each floor. The lastBtnState variable checks the button's previous state, while lastUpdate stores the last time the data was sent to ThingSpeak. I also created an LCD object to show the current floor and fire detection status.

Navarro_EXP7_CPE162P.ino

```
24  void setup() {
25    pinMode(RED_LED, OUTPUT);
26    pinMode(WHITE_LED, OUTPUT);
27    pinMode(buzzer, OUTPUT);
28    pinMode(flamePin, INPUT);
29    pinMode(floorBtn, INPUT_PULLUP);
30
31    lcd.init();
32    lcd.backlight();
33    lcd.clear();
34
35    Serial.begin(115200);
36    WiFi.begin(ssid, password);
37    while (WiFi.status() != WL_CONNECTED) delay(100);
38    ThingSpeak.begin(client);
39
40  }
```

FIGURE 3: The Setup of the Components

Inside the setup() function, I prepared all the hardware parts of the system. I set the flame sensor pin as an input, and the pins for the red LED, white LED, and buzzer as outputs. I also set the floor level button as an input with a pull-up resistor. After that, I initialized the LCD so it can display text, and started the serial monitor to help with debugging. The ESP32 then connects to the Wi-Fi using the SSID and password provided. Once the connection is established, I started the ThingSpeak communication so the system can begin sending data online. This setup ensures that all

components are ready and connected before the main program starts running.

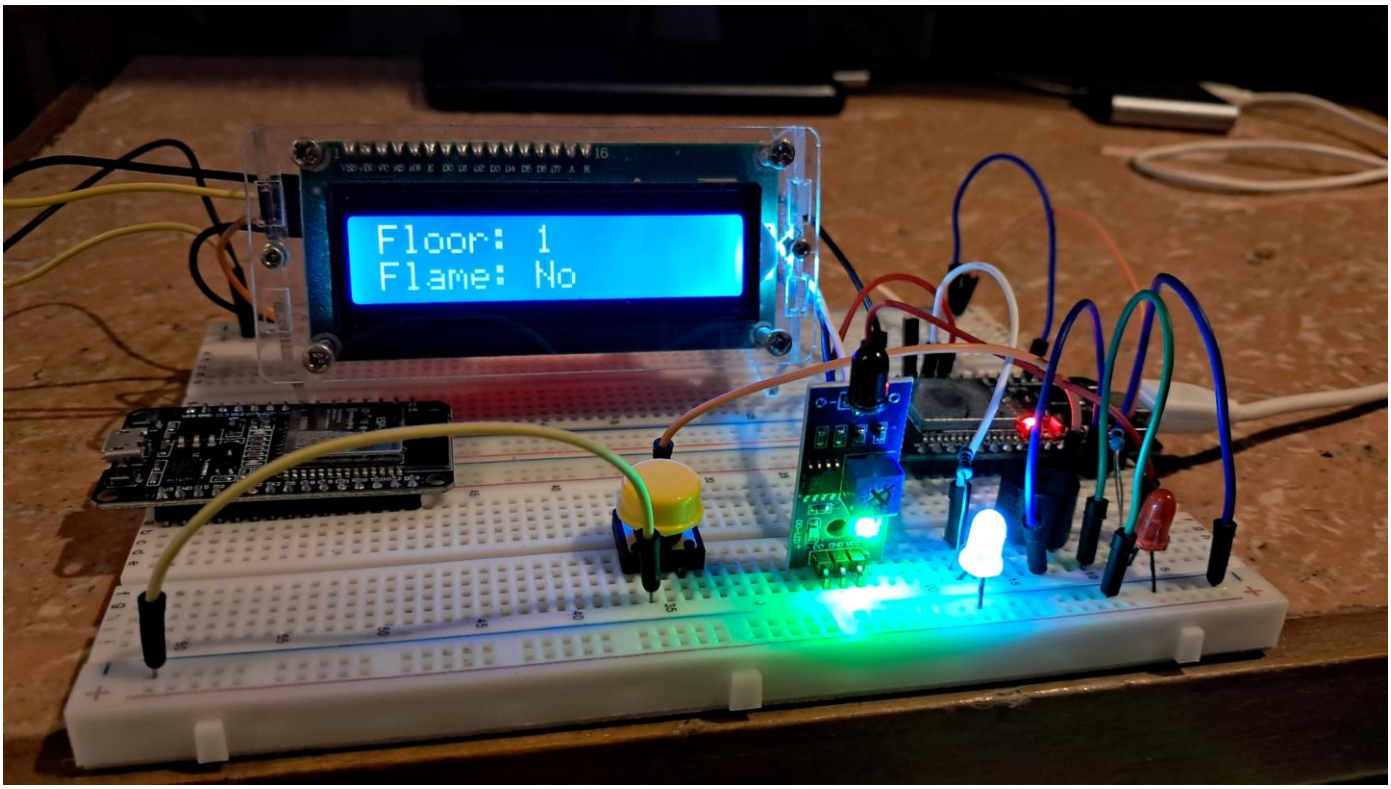
Navarro_EXP7_CPE162P.ino

```
42  void loop() {
43    flame = digitalRead(flamePin);
44
45    if (flame == LOW) {
46      digitalWrite(buzzer, HIGH);
47      digitalWrite(RED_LED, HIGH);
48      digitalWrite(WHITE_LED, LOW);
49      if(floorLevel >= 1 && floorLevel <= 5) {
50        fireCounts[floorLevel]++;
51      }
52    } else {
53       digitalWrite(buzzer, LOW);
54       digitalWrite(RED_LED, LOW);
55       digitalWrite(WHITE_LED, HIGH);
56    }
57
58    // button for the floor levels
59    bool currentBtnState = digitalRead(floorBtn);
60    if (lastBtnState == HIGH && currentBtnState == LOW) {
61      floorLevel = (floorLevel % 5) + 1;
62      delay(200);
63    }
64    lastBtnState = currentBtnState;
65
66    // LCD output
67    lcd.setCursor(0, 0);
68    lcd.print("Floor: ");
69    lcd.print(floorLevel);
70    lcd.print("     ");
71
72    lcd.setCursor(0, 1);
73    lcd.print("Flame: ");
74    lcd.print(flame == LOW ? "YES " : "No ");
```

FIGURE 4: The Loop Function

In the loop() function, the system checks continuously if a flame is detected using the flame sensor. If fire is detected, the buzzer turns on, the red LED lights up, and the white LED turns off. At the same time, the system adds one count to the current floor's fire detection record. If no fire is detected, the buzzer stays off, the red LED is turned off, and the white LED lights up to show everything is normal. The floor level changes whenever the button is pressed, cycling from floor 1 to floor 5. The LCD updates to show the current floor and whether a flame is detected. Every 15 seconds, the system sends updated data to ThingSpeak, including the flame sensor value, current floor, and fire incident counts for each floor. This allows users to monitor fire activity remotely through the internet.

Working Prototype



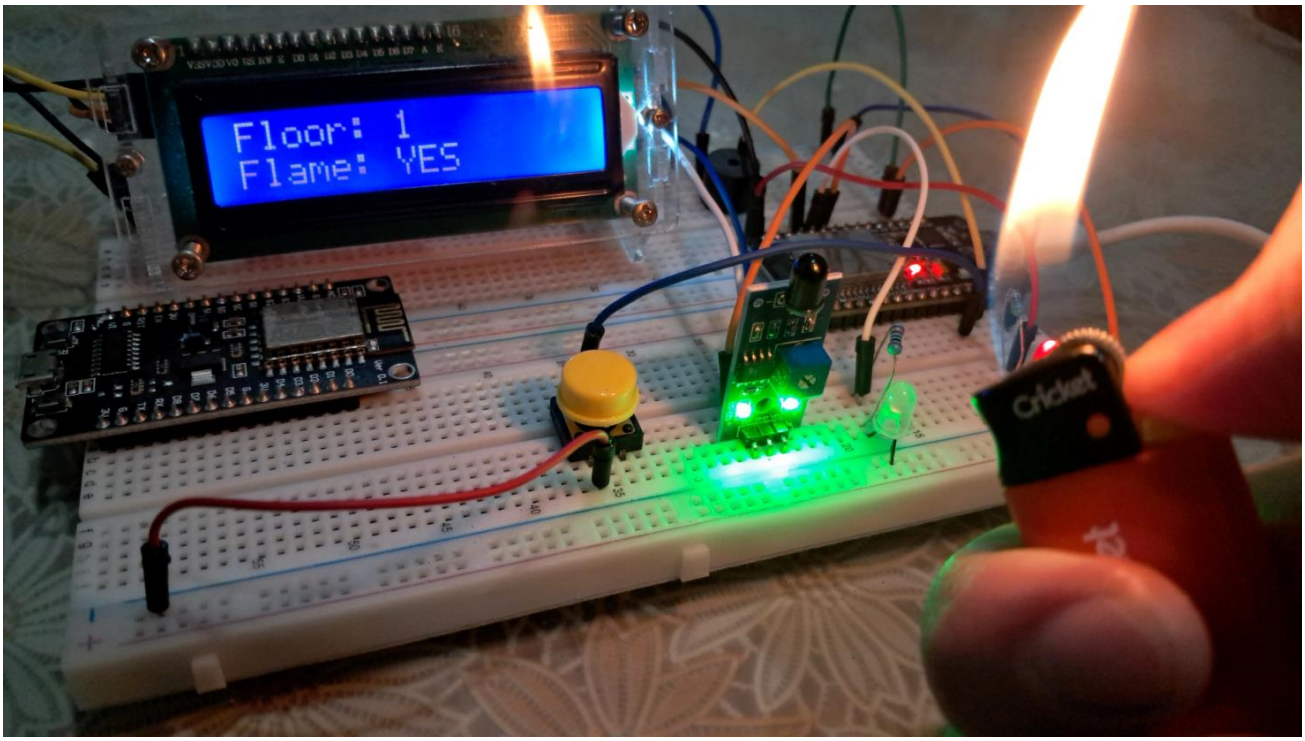
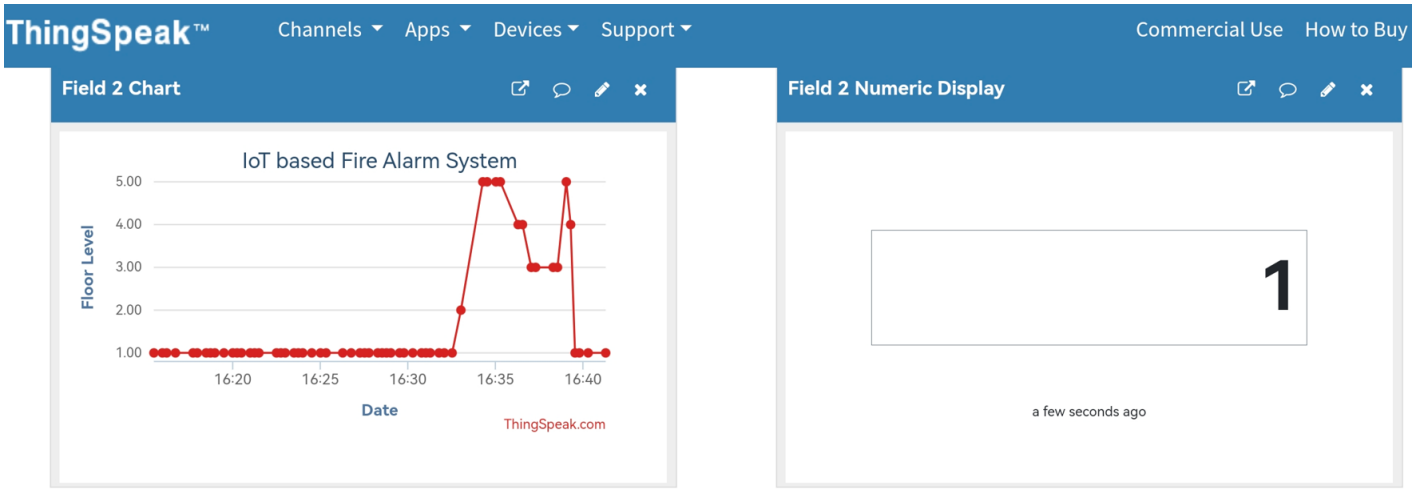


FIGURE 5: Working Prototype

This is the final working prototype of the IoT based fire alarm system experiment, which successfully meets all the requirements.

Interpretations of ThingSpeak's Graphs and Widgets



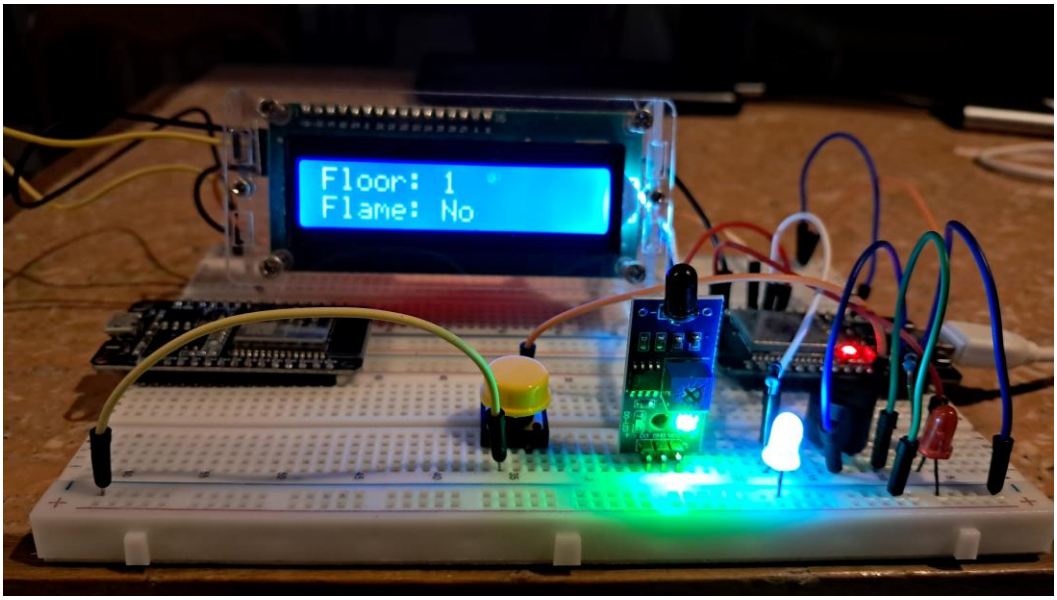


FIGURE 6: ThingSpeak Visualization: Floor Level

This graph and widget provide the data of the current floor the user is currently at. As we can see on the image that at time 16:40 the floor level the user chooses is floor level one.

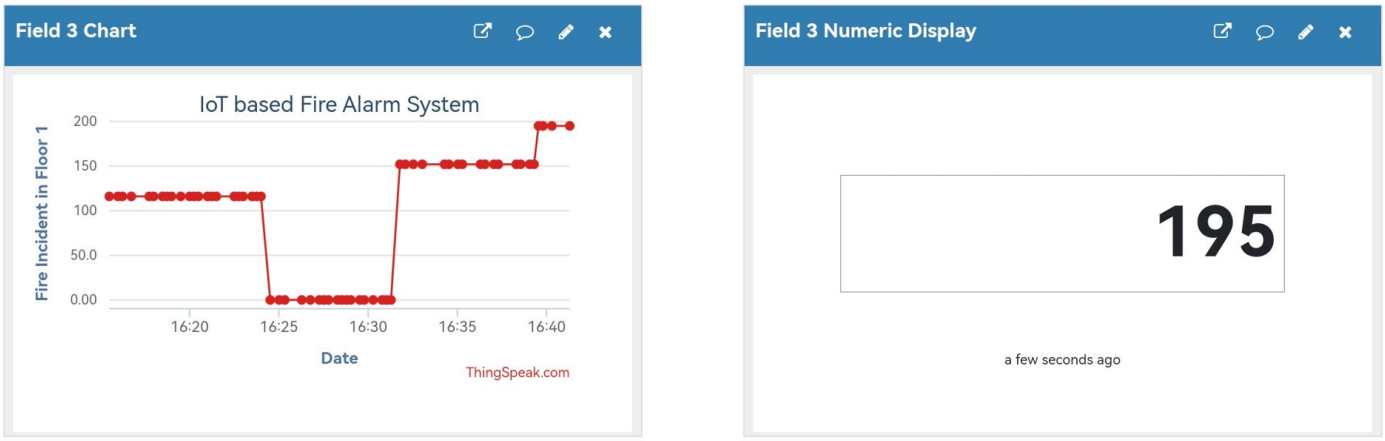


FIGURE 7: ThingSpeak Visualization: Fire Incidents in Floor level 1

The image shows the fire incidents happened at the first floor of the building. The graph provides the specific date and time for each fire incident happened in the floor, it helps track the frequency of the fire. While the widget at the right captures the total value of the number of fires detected in the floor. As we can see that the total number of recorded fire incidents in this floor is 195 which is very high. However, among other floors this floor has the 2nd to the least of fire incidents occurred. The more the fire occurred and detected the larger the value.

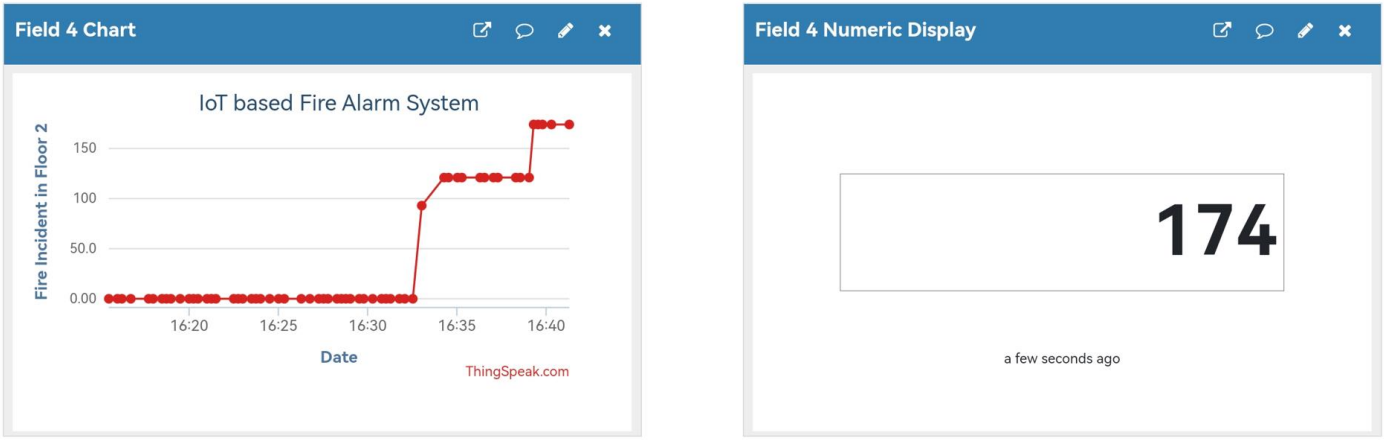


FIGURE 8: ThingSpeak Visualization: Fire Incidents in Floor level 2

For the second floor, shown on the widgets at the right the total value of incident happened in the floor which is a value of 174. The graph provides the specific date and time for each fire incident happened in the floor, it helps track the frequency of the fire. Compared to other floors

this floor has the least fire incidents happened. Despite its rank among other floors this is not a reason to not mitigate the problem in this floor, and the fact that this value is not low. This will serve as an awareness to address whatever fire problem the floor has.

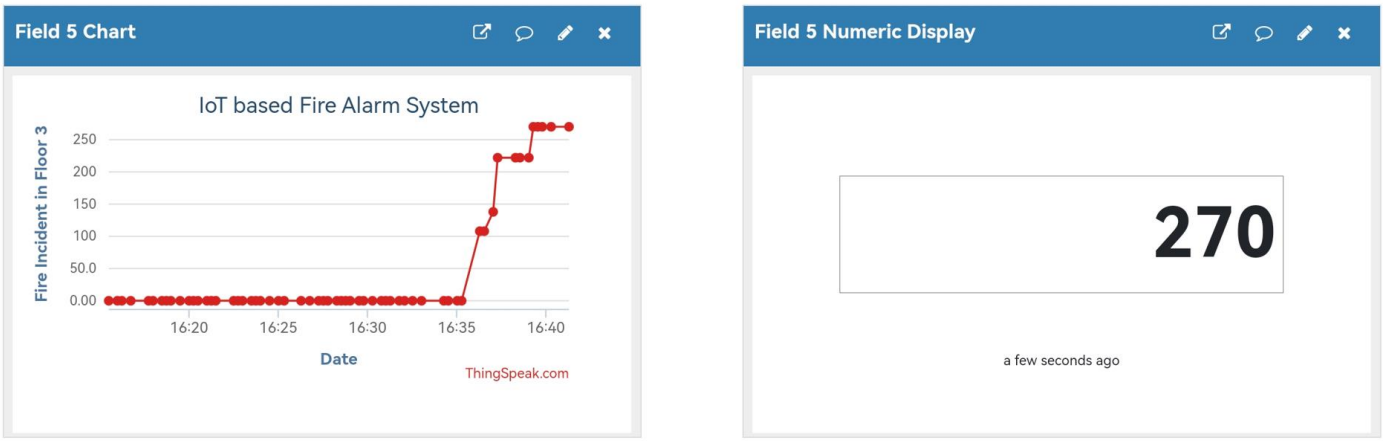


FIGURE 9: ThingSpeak Visualization: Fire Incidents in Floor level 3

The fire incidents happened at floor level 3, we can see the total recorded incidents in this floor with a value of 270. This is the highest recorded fire incidents compared to other floors. This kind of value must be addressed as soon as possible. Assuming that there are many fire hazards present at this floor. With this system, the people will have a real-time feedback and alarm in each floor, monitoring all the status of fires in each floor of the building.

FIGURE 10: ThingSpeak Visualization: Fire Incidents in Floor level 4

At this floor, with use of visualization of data from the flame sensor we can monitor the status of the fire incidents happened in the floor. The widget captured a high value of 264. From the rankings of fire incidents in each floor this has a place in being the second highest fire incident happened.

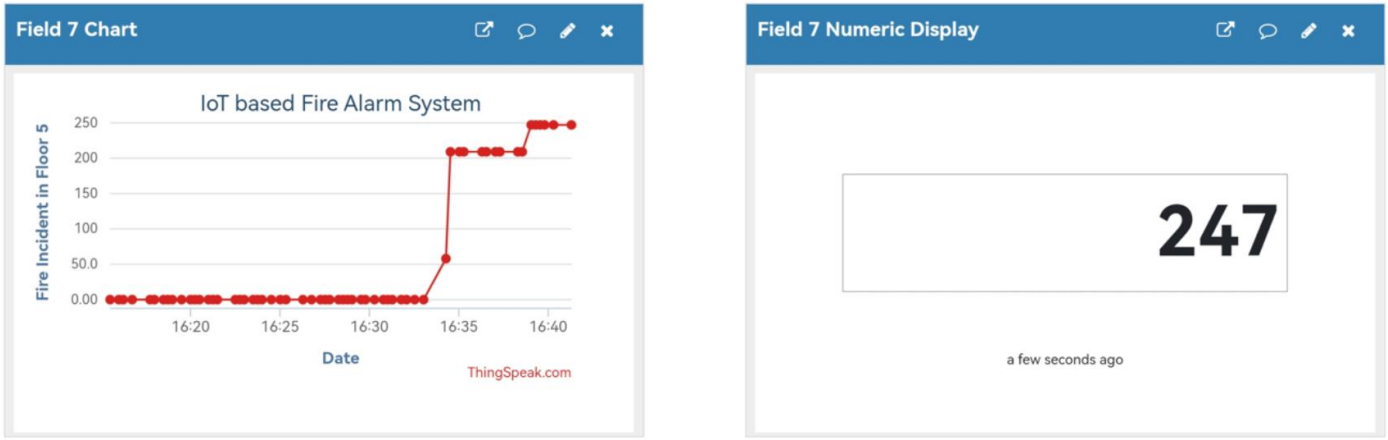


FIGURE 11: ThingSpeak Visualization: Fire Incidents in Floor level 5

This is the highest floor of the building, based on the recorded incidents the widget captured a value of 247. From the rankings of fire incidents in the building this floor has a place in being the third to the highest fire incident happened. We conclude that the more the fire occurred and detected the larger the value. This system can monitor the status of each floor in the building, this can help to solve fire problems in each floor as soon as possible.

CONCLUSION

This project successfully achieved its goal of creating an IoT-based fire alarm monitoring system for a five-story building. By using a flame sensor to detect fire on each floor, the system can give quick alerts through a buzzer and LEDs. The red LED and buzzer turn on when a flame is detected, while the white LED lights up when everything is normal. The LCD displays the current floor and whether fire is detected, making it easy for users to see what's happening at a glance. The button allows users to manually switch between floor levels to simulate different fire situations. All these features work together to help identify which floor is more prone to fire incidents.

With the help of the ESP32 and ThingSpeak, this system is able to send real-time data to the cloud. This means users can monitor fire activity on each floor from anywhere using the internet. The ThingSpeak platform records the number of fire incidents per floor and provides graphs and widgets to visualize the data clearly. From these visualizations, we were able to see that floor 3 had the highest number of fire detections, followed by floors 4 and 5. Meanwhile, floor 2 had the lowest, but it still recorded a significant number. This shows that even floors with fewer incidents must not be ignored. The cloud-based monitoring gives a better understanding of the fire patterns in the building, which can be helpful for planning safety measures.

Building this project helped me learn how to combine sensors, LEDs, an LCD, a buzzer, and a cloud platform into one working system using ESP32 and Arduino IDE. It also taught me the importance of reliable data collection and clear display for monitoring systems. I was able to test different situations, observe the results, and analyze the data in ThingSpeak. The final prototype met all the expected features and provided helpful insights into the fire status of each floor. This kind of

system could be useful in real-life buildings to improve safety and give early warnings during fire emergencies.